



USER_GUIDE



HelixCode User Guide

Getting Started

Installation

Quick Installation

```
# Download and install HelixCode
curl -fsSL https://helixcode.dev/install.sh | bash

# Verify installation
helixcode --version
```

Containerized install (via the `./helix facade`)

Per Constitution Rule 4 + §11.4.76, run the containerized stack through the repo-root `./helix facade` (the host uses podman; direct docker/docker-compose commands are not a supported workflow):

```
./helix start      # boot the containerized stack
./helix status     # check status
./helix logs       # tail logs
./helix stop       # tear down
```

Manual Installation

```
# Clone repository (SSH only – Rule 3 / CONST-038)
git clone git@github.com:HelixDevelopment/HelixCode.git
cd HelixCode

# Build from source (the build target lives in the inner Go
# module)
cd helix_code
make build

# Install globally
sudo cp bin/helixcode /usr/local/bin/
```

Initial Setup

Configuration

Create a configuration file at `~/ .config/helixcode/config.yaml`:

```
server:
  port: 8080
  host: "localhost"

database:
  host: "localhost"
  port: 5432
  user: "helixcode"
  password: "your_password"
  dbname: "helixcode"

workers:
  enabled: true
  auto_install: true
```

Environment Variables

```
# Core System
export HELIX_DATABASE_PASSWORD="your_password"
export HELIX_AUTH_JWT_SECRET="your_jwt_secret"
export HELIX_SERVER_PORT="8080"

# Free AI Provider Tokens (optional)
export GITHUB_TOKEN="your_github_token"           # For GitHub
                                                    Copilot
export OPENROUTER_API_KEY="your_openrouter_key"    # For OpenRouter
export XAI_API_KEY="your_xai_key"                  # For XAI

# Premium AI Provider Tokens (optional)
export ANTHROPIC_API_KEY="sk-ant-your-key"         # For Claude models
export GEMINI_API_KEY="your-gemini-key"            # For Gemini models
export GOOGLE_API_KEY="your-google-key"           # Alternative for
                                                    Gemini
export OPENAI_API_KEY="sk-your-key"                # For OpenAI models
```

Core Concepts

AI Providers

HelixCode supports multiple AI providers with a focus on free and accessible models:

Free Providers (No API Key Required)

- **XAI (Grok):** grok-3-fast-beta, grok-3-mini-fast-beta - Fast and capable models for coding
- **OpenRouter:** openai/gpt-oss-20b:free, meta-llama/llama-3.2-3b-instruct:free - Free models from various providers
- **GitHub Copilot:** gpt-4o, claude-3.5-sonnet, o1 - Free with GitHub subscription
- **Qwen:** OAuth2 authentication available, no API key required for basic usage

Premium Providers

- **OpenAI:** GPT-4, GPT-3.5-turbo with API key
- **Anthropic Claude** : Industry-leading reasoning with advanced features
 - **Models:** Claude 4 Sonnet/Opus, Claude 3.7 Sonnet, Claude 3.5 Sonnet/Haiku
 - **Context:** 200K tokens for all models
 - **Advanced Features:**
 - **Extended Thinking:** Automatic reasoning mode (80% token budget)
 - **Prompt Caching:** Up to 90% cost reduction with multi-layer caching
 - **Tool Caching:** Cache tool definitions across requests
 - **Vision:** Image analysis capabilities
 - **Streaming:** Real-time response generation
- **Google Gemini** : Massive context with multimodal capabilities
 - **Models:** Gemini 2.5 Pro/Flash, Gemini 2.0 Flash, Gemini 1.5 Pro/Flash
 - **Context:** Up to 2M tokens (Gemini 2.5 Pro, 1.5 Pro)
 - **Advanced Features:**
 - **Massive Context:** 2 million token context window for complex codebases
 - **Function Calling:** AUTO/ANY/NONE modes with tool integration
 - **Safety Settings:** Configurable content filtering
 - **Flash Models:** Ultra-fast inference with lite variants
 - **Vision:** Multimodal understanding
 - **Streaming:** Real-time response generation

Distributed Workers

Workers are remote machines that execute tasks. They can be: - **Local Workers:** Same machine as the server - **Remote Workers:** SSH-accessible machines - **Cloud Workers:** Cloud instances (AWS, GCP, Azure)

Tasks

Tasks are units of work that can be: - **Code Generation:** AI-assisted code writing - **Testing:** Automated test execution - **Building:** Compilation and build processes - **Refactoring:** Code improvement and optimization

Projects

Projects organize related tasks and workers: - **Development Projects:** Software development workflows - **Research Projects:** AI research and experimentation - **Infrastructure Projects:** System administration tasks

Basic Usage

Starting the Server

```
# Start the HelixCode server
```

```
helixcode server start
```

```
# Or with custom configuration
```

```
helixcode server start --config /path/to/config.yaml
```

```
# Run in background
```

```
helixcode server start --daemon
```

Adding Workers

Local Worker

```
# Add local worker
```

```
helixcode workers add local --name "local-worker" --capabilities  
    "code-generation,testing"
```

SSH Worker

```
# Add SSH worker
```

```
helixcode workers add ssh \  
    --name "remote-worker" \  
    --ssh-key-path /path/to/key.pem
```

```
--host "192.168.1.100" \  
--port 22 \  
--username "ubuntu" \  
--key-path "~/.ssh/id_rsa" \  
--capabilities "llm-inference,code-generation"
```

Cloud Worker

```
# Add AWS EC2 worker  
helixcode workers add aws \  
  --name "aws-worker" \  
  --instance-id "i-1234567890abcdef0" \  
  --region "us-east-1" \  
  --capabilities "testing,performance"
```

AI Provider Setup

Using Free Providers

```
# Switch to XAI (Grok) - no authentication required  
helixcode config set llm.provider xai
```

```
# Switch to OpenRouter free models  
helixcode config set llm.provider openrouter
```

```
# Use GitHub Copilot (requires GitHub token)  
export GITHUB_TOKEN="ghp_your_token"  
helixcode config set llm.provider copilot
```

```
# Authenticate with Qwen OAuth2  
helixcode auth qwen
```

Using Anthropic Claude

```
# Set up Anthropic API key  
export ANTHROPIC_API_KEY="sk-ant-your-key"
```

```
# Switch to Anthropic provider  
helixcode config set llm.provider anthropic
```

```
# Use specific Claude model  
helixcode config set llm.model "claude-4-sonnet"
```

```
# Enable extended thinking for complex tasks
```

```
helixcode generate --prompt "Think carefully about how to
    optimize this algorithm" \
    --thinking-enabled
```

```
# Use prompt caching for large system prompts
```

```
helixcode generate --system-prompt-file "./
    system_instructions.txt" \
    --cache-enabled \
    --prompt "Add a new feature"
```

```
# Available models:
```

```
# - claude-4-sonnet (most capable, extended thinking)
# - claude-4-opus (highest intelligence)
# - claude-3-7-sonnet (cost-effective)
# - claude-3-5-sonnet-latest (great balance)
# - claude-3-5-haiku-latest (fastest)
```

Using Google Gemini

```
# Set up Gemini API key
```

```
export GEMINI_API_KEY="your-gemini-api-key"
```

```
# Or use Google API key
```

```
export GOOGLE_API_KEY="your-google-api-key"
```

```
# Switch to Gemini provider
```

```
helixcode config set llm.provider gemini
```

```
# Use massive context model for large codebases
```

```
helixcode config set llm.model "gemini-2.5-pro"
```

```
# Analyze entire large codebase (2M tokens)
```

```
helixcode analyze --model "gemini-2.5-pro" --recursive ./large-
    project
```

```
# Use Flash models for fast responses
```

```
helixcode generate --model "gemini-2.5-flash-lite" \
    --prompt "Quick code snippet for authentication"
```

```
# Configure safety settings
```

```
helixcode config set llm.gemini.safety.harassment
    "BLOCK_ONLY_HIGH"
```

```
# Enable function calling
```

```
helixcode generate --model "gemini-2.5-flash" \
    --tools "./tools.json" \
```

```
--tool-mode "AUTO"
```

```
# Available models:  
# - gemini-2.5-pro (2M context, most capable)  
# - gemini-2.5-flash (1M context, fast)  
# - gemini-2.5-flash-lite (fastest)  
# - gemini-2.0-flash (balanced)  
# - gemini-1.5-pro (2M context, stable)  
# - gemini-1.5-flash (affordable)
```

Checking Provider Status

```
# List available providers  
helixcode llm providers list
```

```
# Check current provider health  
helixcode llm provider health
```

```
# Test current provider  
helixcode llm test "Hello, can you help me write a Go function?"
```

Managing Workers

```
# List all workers  
helixcode workers list
```

```
# Get worker details  
helixcode workers info worker-id
```

```
# Remove worker  
helixcode workers remove worker-id  
✖
```

```
# Check worker health  
helixcode workers health
```

Task Management

Creating Tasks

Code Generation Task

```
# Generate code from prompt  
helixcode tasks create code-generation \  
  --prompt "Create a REST API in Go with authentication" \  
  --
```

```
--language "go" \  
--framework "gin" \  
--output-dir "./generated-api"
```

Testing Task

```
# Run comprehensive tests  
helixcode tasks create testing \  
  --project-path "./my-project" \  
  --test-type "unit,integration,e2e" \  
  --coverage-threshold 80
```

Building Task

```
# Build project  
helixcode tasks create building \  
  --project-path "./my-project" \  
  --build-type "release" \  
  --platforms "linux, windows, darwin"
```

Task Monitoring

```
# List all tasks  
helixcode tasks list
```

```
# Get task status  
helixcode tasks status task-id
```

```
# View task logs  
helixcode tasks logs task-id
```

```
# Cancel task  
helixcode tasks cancel task-id  
\  
# Retry failed task  
helixcode tasks retry task-id
```

Advanced Features

Work Preservation

HelixCode automatically preserves work during: - **Worker Failures:** Tasks automatically reassigned - **Network Issues:** Checkpoints saved periodically - **System Restarts:** State restored from checkpoints

Checkpoint Management

List checkpoints for a task

```
helixcode tasks checkpoints task-id
```

Restore from checkpoint

```
helixcode tasks restore task-id --checkpoint checkpoint-id
```

Manual checkpoint creation

```
helixcode tasks checkpoint task-id
```

Distributed Development

Planning Mode

Analyze project and create development plan

```
helixcode workflow planning \  
  --project-path "./my-project" \  
  --requirements "high-performance,scalable,microservices"
```

Building Mode

Distributed compilation

```
helixcode workflow building \  
  --project-path "./my-project" \  
  --workers 5 \  
  --cache-enabled true
```

Testing Mode

Parallel test execution

```
helixcode workflow testing \  
  --project-path "./my-project" \  
  --test-suites "unit,integration,performance" \  
  --parallel-workers 3
```

Refactoring Mode

AI-assisted refactoring

```
helixcode workflow refactoring \  
  --project-path "./my-project" \  
  --targets "performance,readability,security" \  
  --safety-checks true
```

MCP Integration

Adding MCP Servers

```
# Add stdio MCP server
helixcode mcp add developer \
  --type "stdio" \
  --command "mcp-developer-server" \
  --args "--verbose"

# Add HTTP MCP server
helixcode mcp add memory \
  --type "http" \
  --url "https://memory-server.example.com/mcp" \
  --headers "Authorization=Bearer ${TOKEN}"
```

Managing MCP Tools

```
# List available tools
helixcode mcp tools

# Execute tool
helixcode mcp execute tool-name --parameters '{"param": "value"}'

# Monitor tool usage
helixcode mcp stats
```

Customization

Themes and Appearance

```
# List available themes
helixcode themes list

# Set theme
helixcode themes set "dark"

# Create custom theme
helixcode themes create my-theme --colors '{"primary":
  "#C2E95B"}'
```

Configuration Profiles

```
# Create development profile
helixcode config profile create dev \
  --workers 2 \
  --llm-provider local \
  --notifications disabled

# Switch to production profile
helixcode config profile use prod

# List profiles
helixcode config profile list
```

API Usage

REST API

Authentication

```
# Get authentication token
curl -X POST http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username": "admin", "password": "password"}'
```

Task Management

```
# Create task via API
curl -X POST http://localhost:8080/api/v1/tasks \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "type": "code-generation",
    "payload": {
      "prompt": "Create a REST API",
      "language": "go"
    }
  }'
```

Worker Management

```
# List workers via API
curl -X GET http://localhost:8080/api/v1/workers \
  -H "Authorization: Bearer $TOKEN"
```

WebSocket API

Real-time Updates

```
const ws = new WebSocket('ws://localhost:8080/ws');
ws.onmessage = (event) => {
  ⚡ const data = JSON.parse(event.data);
  console.log('Task update:', data);
};
```

Performance Optimization

Worker Configuration

Optimize for Code Generation

```
workers:
  code-gen-worker:
    capabilities: ["code-generation"]
    resources:
      cpu: 8
      memory: 16GB
      gpu: true
    optimization:
      batch_size: 10
      cache_enabled: true
```

Optimize for Testing

```
workers:
  test-worker:
    capabilities: ["testing"]
    resources:
      cpu: 4
      memory: 8GB
    optimization:
      parallel_tests: 8
      test_timeout: 300
```

Task Optimization

Batch Processing

```
# Process multiple files in batch
helixcode tasks create code-generation \
  --batch "./src/**/*.go" \
  --pattern "*_test.go" \
  --workers 3
```

Caching Strategy

```
# Enable build caching
helixcode config set build.cache.enabled true
helixcode config set build.cache.ttl "24h"
🔒

# Clear cache
helixcode cache clear
```

Security Best Practices

Authentication

- Use strong passwords and JWT secrets
- Enable multi-factor authentication
- Regularly rotate API keys
- Use environment variables for secrets

Network Security

- Use HTTPS in production
- Configure firewall rules
- Use VPN for remote workers
- Monitor network traffic

Worker Security

- Use SSH keys instead of passwords
- Regularly update worker software
- Monitor worker activity
- Implement access controls

Troubleshooting

Common Issues

Worker Connection Issues

Test SSH connection

```
helixcode workers test worker-id
```

Check worker logs

```
helixcode workers logs worker-id
```

Restart worker

```
helixcode workers restart worker-id
```

Task Failures

Get detailed error information

```
helixcode tasks debug task-id
```

View task execution history

```
helixcode tasks history task-id
```

Reset task state

```
helixcode tasks reset task-id
```

Performance Issues

Monitor system resources

```
helixcode system stats
```

Check worker load

```
helixcode workers load
```

Optimize configuration

```
helixcode config optimize
```

Logs and Diagnostics

View server logs

```
helixcode server logs
```

Get system diagnostics

```
helixcode system diagnostics
```



Generate debug report
helixcode debug report

Additional Resources

- **Documentation:** <https://docs.helixcode.dev>
- **Community Forum:** <https://community.helixcode.dev>
- **GitHub Repository:** <https://github.com/helixcode/helixcode>
- **API Reference:** <https://api.helixcode.dev>



User Guide Version: 1.1.0 **Last Updated:** 2025-11-05 **Support:**
support@helixcode.dev

Recent Updates

Version 1.1.0 (2025-11-05)

New AI Providers: - **Anthropic Claude:** Direct API integration with extended thinking, prompt caching, and tool caching - **Google Gemini:** Full API support with 2M token context windows and function calling

Advanced Features: - Extended thinking for complex reasoning tasks (Anthropic) - Multi-layer prompt caching with 90% cost savings (Anthropic) - Massive 2M token context for entire codebase analysis (Gemini) - Enhanced function calling with AUTO/ANY/NONE modes (Gemini) - Vision capabilities for both providers - Real-time streaming for both providers

Test Coverage: - 100% test coverage across unit, automation, and e2e tests - 73+ comprehensive test functions - All tests passing with 100% success rate